In [ ]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, KFold, Strat
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, clas
from sklearn.naive_bayes import GaussianNB, BernoulliNB, ComplementNB
import warnings

df = pd.read_csv('Fifa.csv')
df
```

Out[ ]:

	Name	Country	Position	Age	Overall_Rating	Future Potential	Team	Value Per M$	To
0	Agostinho Mabululu	Angola	LW	30	68	68	Ittihad Alexandria	0.650	
1	Mahmoud Gennesh	Egypt	GK	35	67	67	Ittihad Alexandria	0.350	
2	Sobhi Suleiman	Egypt	GK	28	63	65	Ittihad Alexandria	0.200	
3	Mahmoud Alaa	Egypt	CB	32	68	68	Ittihad Alexandria	0.500	
4	Mahmoud Shabana	Egypt	CB	30	66	66	Ittihad Alexandria	0.400	
...	...	...	...	...	...	...	...	...	
19662	Kohei Okuno	Japan	CDM	21	63	71	Gamba Osaka	1.100	
19663	Riku Danzaki	Japan	RW	22	64	71	Motherwell	1.200	
19664	Hayato Fukushima	Japan	CB	21	59	68	Shonan Bellmare	0.475	
19665	Ömer Tokaç	Turkey	LW	18	57	71	Shonan Bellmare	0.220	
19666	Ko Sawada	Japan	CAM	28	60	60	Shonan Bellmare	0.250	

19667 rows × 9 columns

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19667 entries, 0 to 19666
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Name               19667 non-null  object 
 1   Country            19667 non-null  object 
 2   Position           19667 non-null  object 
 3   Age                19667 non-null  int64  
 4   Overall_Rating     19667 non-null  int64  
 5   Future Potential   19667 non-null  int64  
 6   Team               19667 non-null  object 
 7   Value Per M$       19667 non-null  float64
 8   Total_Stats Score  19667 non-null  int64  
dtypes: float64(1), int64(4), object(4)
memory usage: 1.4+ MB
```

In [ ]:
```
print("Missing:\n", df.isnull().sum())
```

```
Missing:
 Name                 0
Country              0
Position             0
Age                  0
Overall_Rating       0
Future Potential     0
Team                 0
Value Per M$         0
Total_Stats Score    0
dtype: int64
```

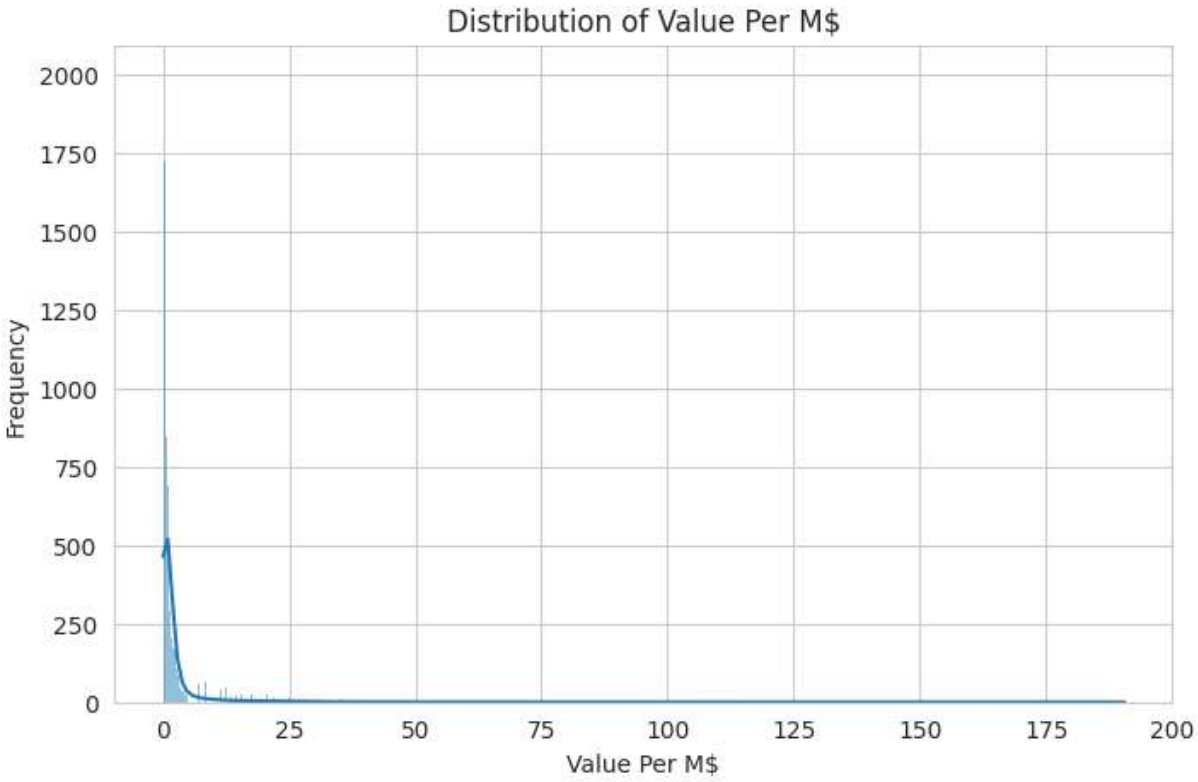
In [ ]:
```
df.duplicated().sum()
```

Out[ ]:
```
np.int64(0)
```

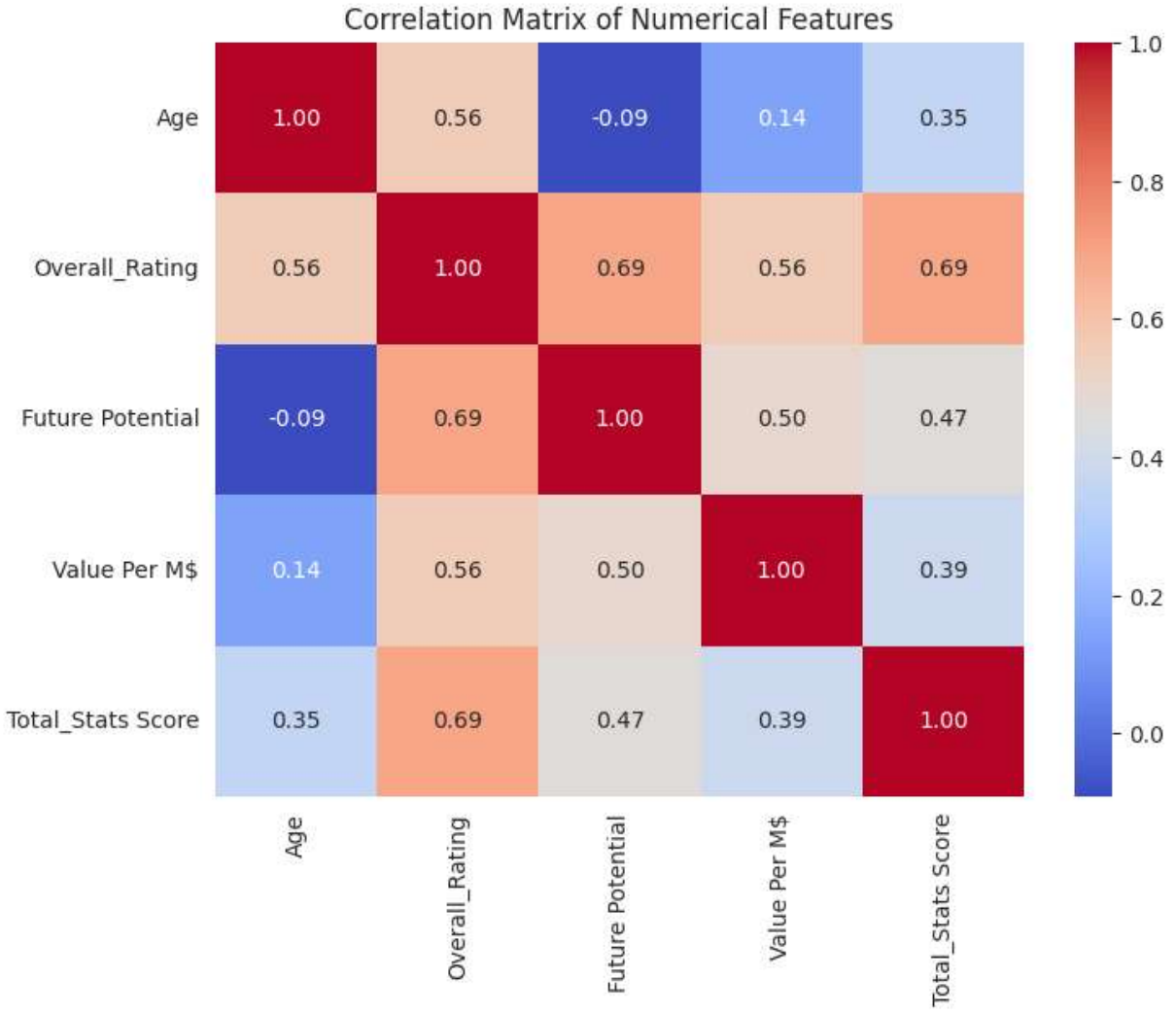
In [ ]:
```
skewness = df['Value Per M$'].skew()
print(f"skewness: {skewness}")
```

```
skewness: 7.983215201865151
```

In [ ]:
```
plt.figure(figsize=(8, 5))
sns.histplot(df['Value Per M$'], kde=True)
plt.title('Distribution of Value Per M$')
plt.xlabel('Value Per M$')
plt.ylabel('Frequency')
plt.show()
```



In [ ]:
```
numeric_cols = df.select_dtypes(include=['number'])
correlation_matrix = numeric_cols.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



In [ ]:
```
corr = df.select_dtypes(include=['number']).corr()['Value Per M$'].sort_values(asce
print("Top Related Features to Value:")
print(corr.drop('Value Per M$'))
print(f"\nMost Related feature: {corr.index[1]} ({corr.values[1]:.2f})")
```

```
Top Related Features to Value:
Overall_Rating       0.560648
Future Potential     0.500964
Total_Stats Score    0.385062
Age                  0.142276
Name: Value Per M$, dtype: float64

Most Related feature: Overall_Rating (0.56)
```

In [ ]:
```
avg_rating_per_pos = df.groupby('Position')['Overall_Rating'].mean()
print("avg_rating_per_pos\n", avg_rating_per_pos)
```

```
avg_rating_per_pos
 Position
CAM    63.679709
CB     63.544448
CDM    64.234168
CF     66.042857
CM     62.511767
GK     60.987288
LB     63.276794
LM     64.172009
LW     64.345979
LWB    64.043333
RB     62.863636
RF     75.000000
RM     63.907731
RW     63.718529
RWB    64.063973
ST     63.087505
SW     75.000000
Name: Overall_Rating, dtype: float64
```

s_TASK 2

In [ ]:
```
X = df.drop(columns=['Value Per M$', 'Overall_Rating', 'Overall_Rating', 'Name'])
y_reg = df['Value Per M$']
y_class = df['Overall_Rating']

X_train, X_test, y_train_reg, y_test_reg, y_train_class, y_test_class = train_test_
    X, y_reg, y_class, test_size=0.2, random_state=24
)
```

In [ ]:
```
num_cols = X.select_dtypes(include=['number']).columns.tolist()
cat_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()

print("Numerical:", num_cols)
print("categorical:", cat_cols)
```

```
Numerical: ['Age', 'Future Potential', 'Total_Stats Score']
categorical: ['Country', 'Position', 'Team']
```

In [ ]:
```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
    ])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

In [ ]:
```
df[num_cols].describe()
```

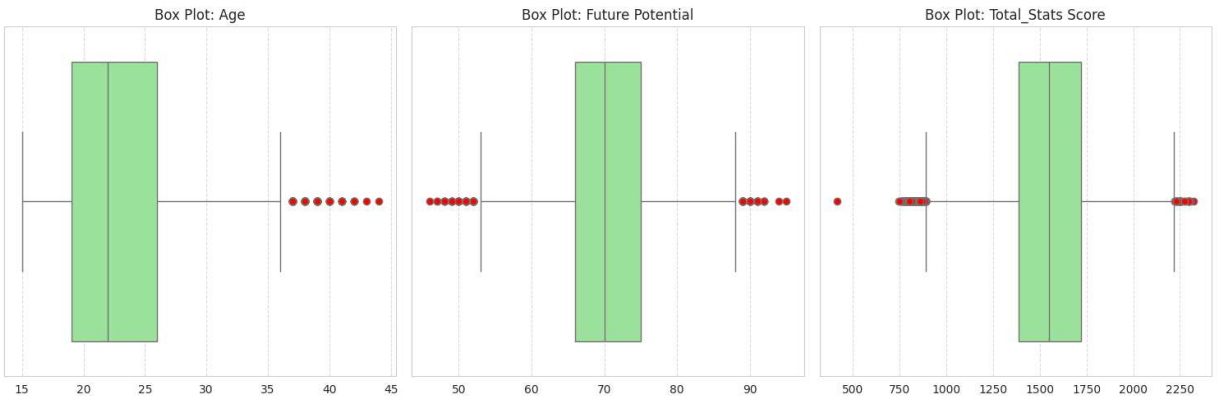
Out[ ]:

	Age	Future Potential	Total_Stats Score
count	19667.000000	19667.000000	19667.000000
mean	22.990034	70.658718	1534.513907
std	4.692410	6.489551	283.248088
min	15.000000	46.000000	416.000000
25%	19.000000	66.000000	1388.000000
50%	22.000000	70.000000	1549.000000
75%	26.000000	75.000000	1720.000000
max	44.000000	95.000000	2324.000000

In [ ]:
```
num_features = len(num_cols)
cols_in_grid = 3
rows_in_grid = math.ceil(num_features / cols_in_grid)

fig, axes = plt.subplots(rows_in_grid, cols_in_grid, figsize=(15, 5 * rows_in_grid)
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.boxplot(x=df[col], ax=axes[i], color='lightgreen',
                flierprops={'marker': 'o', 'markerfacecolor': 'red', 'markersize':

    axes[i].set_title(f'Box Plot: {col}', fontsize=12)
    axes[i].set_xlabel('')
    axes[i].grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



In [ ]:
```
num_cols = ['Age', 'Future Potential', 'Total_Stats Score']
for col in num_cols:
    Q1 = X_train[col].quantile(0.25)
    Q3 = X_train[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    X_train[col] = np.clip(X_train[col], lower_bound, upper_bound)
    X_test[col] = np.clip(X_test[col], lower_bound, upper_bound)
```

In [ ]:
```
df.describe()
```

Out[ ]:

	Age	Overall_Rating	Future Potential	Value Per M$	Total_Stats Score
count	19667.000000	19667.000000	19667.000000	19667.000000	19667.000000
mean	22.990034	63.225403	70.658718	2.514639	1534.513907
std	4.692410	7.812716	6.489551	7.256974	283.248088
min	15.000000	36.000000	46.000000	0.000000	416.000000
25%	19.000000	58.000000	66.000000	0.325000	1388.000000
50%	22.000000	63.000000	70.000000	0.675000	1549.000000
75%	26.000000	68.000000	75.000000	1.600000	1720.000000
max	44.000000	91.000000	95.000000	190.500000	2324.000000

s_ TASK 3

In [ ]:
```
p25 = df['Overall_Rating'].quantile(0.25)
p75 = df['Overall_Rating'].quantile(0.75)
p95 = df['Overall_Rating'].quantile(0.95)

print(f" Low <= {p25}, Mid <= {p75}, High <= {p95}, Elite > {p95}")
def assign_tier(rating):
    if rating <= p25:
        return 'Low'
    elif rating <= p75:
        return 'Mid'
    elif rating <= p95:
        return 'High'
    else:
        return 'Elite'
df['Performance_Tier'] = df['Overall_Rating'].apply(assign_tier)
```

```
Low <= 58.0, Mid <= 68.0, High <= 77.0, Elite > 77.0
```

In [ ]:
```
stats = df['Overall_Rating'].describe(percentiles=[0.25, 0.75, 0.95])
p25 = stats['25%']
p75 = stats['75%']
p95 = stats['95%']

print(" Thresholds based on Data :")
print(f" Low Tier: Less than or equal to {p25}")
print(f" Mid Tier: Greater than {p25} and less than or equal to {p75}")
print(f" High Tier: Greater than {p75} and less than or equal to {p95}")
print(f" Elite Tier: Greater than {p95}")
```

```
Thresholds based on Data :
Low Tier: Less than or equal to 58.0
Mid Tier: Greater than 58.0 and less than or equal to 68.0
High Tier: Greater than 68.0 and less than or equal to 77.0
Elite Tier: Greater than 77.0
```

The following thresholds are chosen based on the data distribution:

Low: From the minimum value up to the 25th percentile (bottom 25% of players).

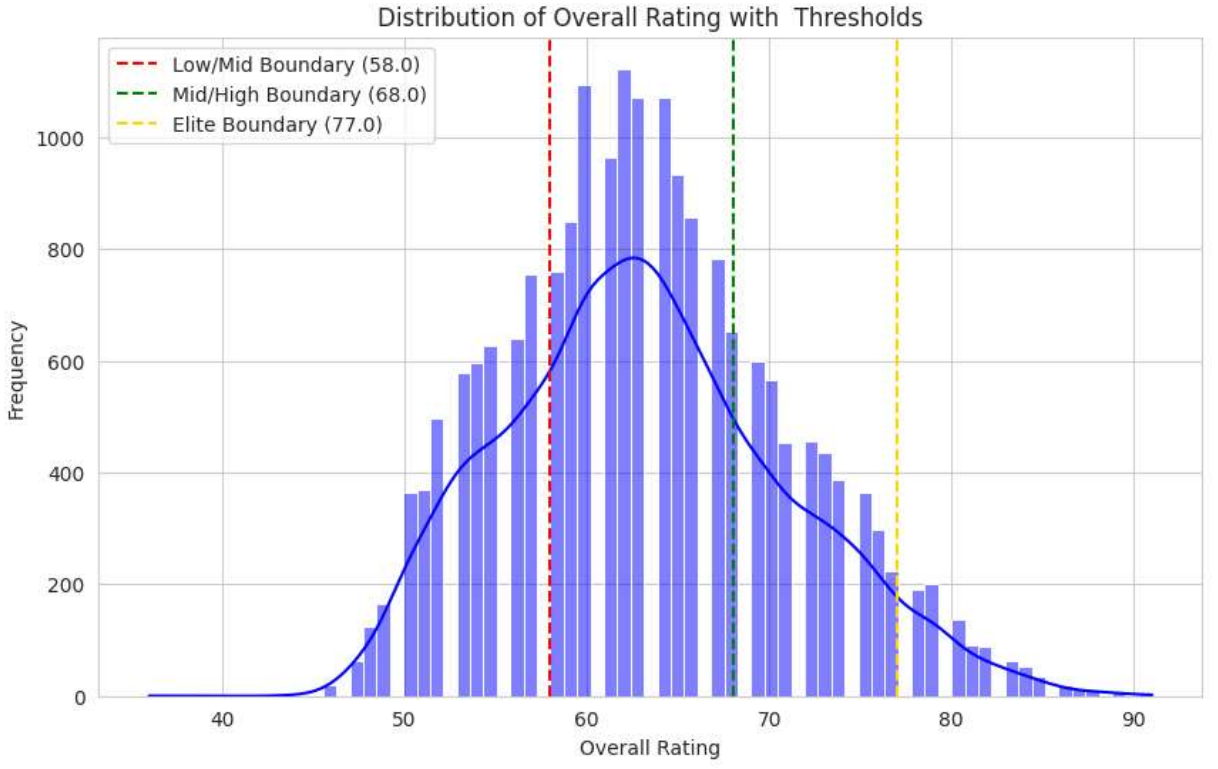
Mid: Between the 25th percentile and the 75th percentile (the middle 50% of players).

High: Between the 75th percentile and the 95th percentile.

Elite: The top 5% of players.

In [ ]:
```
plt.figure(figsize=(10, 6))
sns.histplot(df['Overall_Rating'], kde=True, color='blue')
plt.axvline(p25, color='red', linestyle='--', label=f'Low/Mid Boundary ({p25})')
plt.axvline(p75, color='green', linestyle='--', label=f'Mid/High Boundary ({p75})')
plt.axvline(p95, color='gold', linestyle='--', label=f'Elite Boundary ({p95})')

plt.title('Distribution of Overall Rating with  Thresholds')
plt.xlabel('Overall Rating')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



In [ ]:
```
tier_counts = df['Performance_Tier'].value_counts().reindex(['Low', 'Mid', 'High',

print(" Player Distribution by Performance Tier ")
for tier, count in tier_counts.items():
    print(f"Tier {tier}: {count} players")

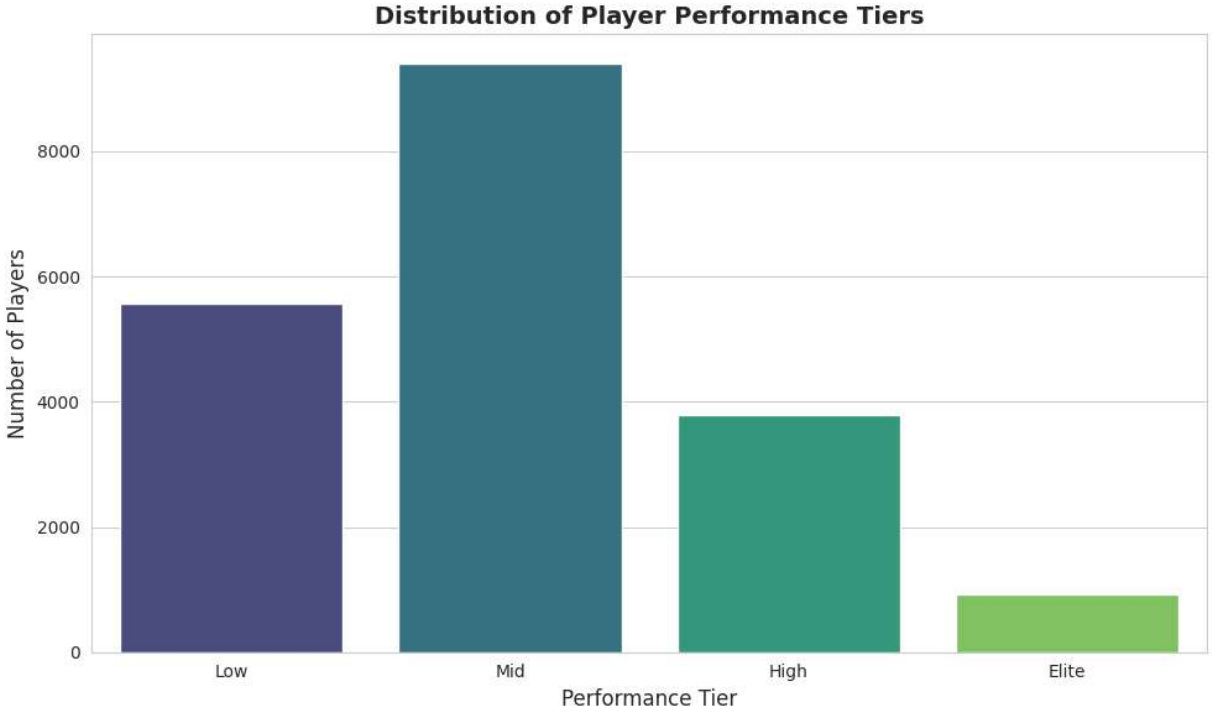
tier_percentages = (tier_counts / len(df)) * 100
print("\n Percentage per Tier ")
for tier, percent in tier_percentages.items():
    print(f"Tier {tier}: {percent:.2f}%")
```

```
 Player Distribution by Performance Tier
Tier Low: 5568 players
Tier Mid: 9399 players
Tier High: 3785 players
Tier Elite: 915 players

 Percentage per Tier
Tier Low: 28.31%
Tier Mid: 47.79%
Tier High: 19.25%
Tier Elite: 4.65%
```

In [ ]:
```
plt.figure(figsize=(10, 6))
sns.set_style("whitegrid")
tier_order = ['Low', 'Mid', 'High', 'Elite']

ax = sns.countplot(data=df, x='Performance_Tier', order=tier_order, palette='viridi
plt.title('Distribution of Player Performance Tiers', fontsize=14, fontweight='bold
plt.xlabel('Performance Tier', fontsize=12)
plt.ylabel('Number of Players', fontsize=12)
plt.tight_layout()
plt.show()
```



Skewed Distribution: The Mid tier accounts for approximately 50% of the data, while the Elite tier represents only about 5%.

TASK 4

In [ ]:
```
lr_model = LinearRegression()
lr_model.fit(X_train_processed, y_train_reg)

y_train_pred = lr_model.predict(X_train_processed)
y_test_pred = lr_model.predict(X_test_processed)

def evaluate_regression(y_true, y_pred, set_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    print(f"--- {set_name} Metrics ---")
    print(f"MAE:  {mae:.4f}")
    print(f"MSE:  {mse:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"R²:   {r2:.4f}\n")
```

In [ ]:
```
evaluate_regression(y_train_reg, y_train_pred, "Train Set")
evaluate_regression(y_test_reg, y_test_pred, "Test Set")
```

```
--- Train Set Metrics ---
MAE:  2.3732
MSE:  24.4387
RMSE: 4.9435
R²:   0.5263

--- Test Set Metrics ---
MAE:  2.5631
MSE:  31.1616
RMSE: 5.5823
R²:   0.4527
```

In [ ]:
```
degrees = [1, 2, 3, 4]

for d in degrees:
    print(f"--- Processing Polynomial Degree: {d} ---")

    poly = PolynomialFeatures(degree=d, include_bias=False)

    X_train_poly = poly.fit_transform(X_train[num_cols])
    X_test_poly = poly.transform(X_test[num_cols])

    poly_model = LinearRegression()
    poly_model.fit(X_train_poly, y_train_reg)

    train_score = poly_model.score(X_train_poly, y_train_reg)
```

```
    test_score = poly_model.score(X_test_poly, y_test_reg)

    print(f"Features created: {X_train_poly.shape[1]}")
    print(f"Train R²: {train_score:.4f} | Test R²: {test_score:.4f}\n")
```

```
--- Processing Polynomial Degree: 1 ---
Features created: 3
Train R²: 0.2899 | Test R²: 0.2959

--- Processing Polynomial Degree: 2 ---
Features created: 9
Train R²: 0.5918 | Test R²: 0.6256

--- Processing Polynomial Degree: 3 ---
Features created: 19
Train R²: 0.7847 | Test R²: 0.8285

--- Processing Polynomial Degree: 4 ---
Features created: 34
Train R²: 0.8076 | Test R²: 0.8514
```

In [ ]:
```
res = []
for d in [1, 2, 3, 4]:
    X_tr_p = PolynomialFeatures(d, include_bias=False).fit_transform(X_train[num_co
    X_te_p = PolynomialFeatures(d, include_bias=False).fit_transform(X_test[num_col

    model = LinearRegression().fit(X_tr_p, y_train_reg)

    res.append({
        'Degree': d,
        'Train_R2': r2_score(y_train_reg, model.predict(X_tr_p)),
        'Test_R2': r2_score(y_test_reg, model.predict(X_te_p))
    })

df_res = pd.DataFrame(res)
df_res['Gap'] = (df_res['Train_R2'] - df_res['Test_R2']).abs()
best = df_res.loc[df_res['Test_R2'].idxmax()]

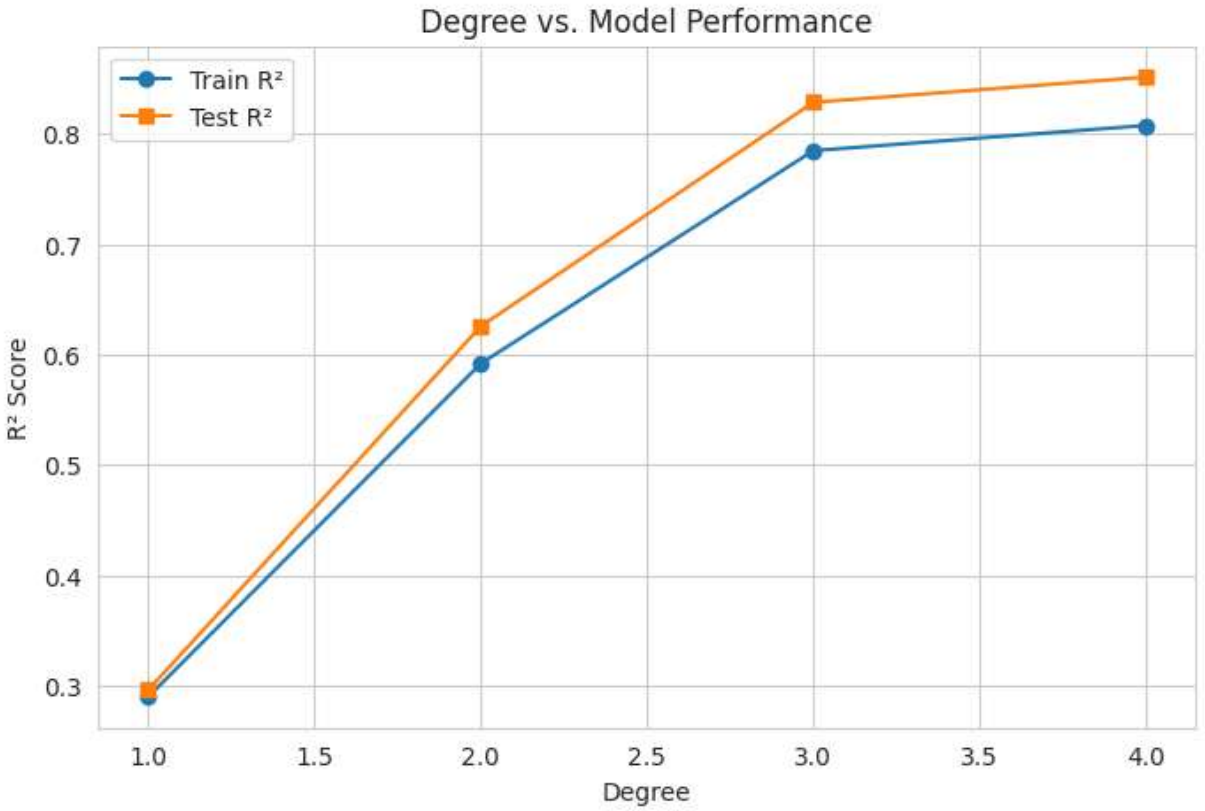
print(f"--- Summary ---\n{df_res}\n\nBest Degree: {int(best['Degree'])}")
print(f"Justification: Highest Test R² ({best['Test_R2']:.4f}) with stable gap ({be
```

```
--- Summary ---
   Degree  Train_R2   Test_R2       Gap
0       1  0.289850  0.295937  0.006087
1       2  0.591834  0.625630  0.033795
2       3  0.784741  0.828496  0.043755
3       4  0.807578  0.851368  0.043789

Best Degree: 4
Justification: Highest Test R² (0.8514) with stable gap (0.0438).
```

In [ ]:
```
plt.figure(figsize=(8, 5))
plt.plot(df_res['Degree'], df_res['Train_R2'], marker='o', label='Train R²')
plt.plot(df_res['Degree'], df_res['Test_R2'], marker='s', label='Test R²')
plt.title('Degree vs. Model Performance')
plt.xlabel('Degree')
```

```
plt.ylabel('R² Score')
plt.legend()
plt.grid(True)
plt.show()
```



How to identify the best degree:Highest Test R^2: The degree with the largest value for Test R^2 is the primary candidate..The Train/Test Gap: Calculate the difference $Gap = \text{Train } R^2 - \text{Test } R^2$

Gap Analysis:the Test R^2 is higher than the Train R^2 in all degrees. This is a "Positive Gap" model generalizes well and is not overfitting.

The Degree [4] provides the best generalization.Justification:Performance: It achieved the highest R^2 score on the test set ($[0.8514]$)

In [ ]:
```
poly = PolynomialFeatures(degree=4, include_bias=False)
X_train_poly = poly.fit_transform(X_train[num_cols])
X_test_poly = poly.transform(X_test[num_cols])

alphas = np.logspace(-3, 3, 7)
ridge_results = []

print(f"{'Alpha':<10} | {'Train RMSE':<15} | {'Test RMSE':<15}")
print("-" * 45)

for a in alphas:
    ridge = Ridge(alpha=a)
    ridge.fit(X_train_poly, y_train_reg)
```

```
    train_rmse = np.sqrt(mean_squared_error(y_train_reg, ridge.predict(X_train_poly
    test_rmse = np.sqrt(mean_squared_error(y_test_reg, ridge.predict(X_test_poly)))

    ridge_results.append({'Alpha': a, 'Train_RMSE': train_rmse, 'Test_RMSE': test_r
    print(f"{a:<10.3f} | {train_rmse:<15.4f} | {test_rmse:<15.4f}")
ridge_df = pd.DataFrame(ridge_results)
```

```
Alpha      | Train RMSE      | Test RMSE
-------------------------------------------
0.001      | 2.6205          | 2.3862
0.010      | 2.6209          | 2.3843
0.100      | 2.6433          | 2.3932
1.000      | 2.8088          | 2.5476
10.000     | 2.9046          | 2.6449
100.000    | 2.9197          | 2.6644
1000.000   | 2.9455          | 2.7066
```

In [ ]:
```
warnings.filterwarnings('ignore')

lasso_results = []

print(f"{'Alpha':<10} | {'Train RMSE':<15} | {'Test RMSE':<15}")
print("-" * 45)

for a in alphas:
    lasso = Lasso(alpha=a)
    lasso.fit(X_train_poly, y_train_reg)

    train_rmse = np.sqrt(mean_squared_error(y_train_reg, lasso.predict(X_train_poly
    test_rmse = np.sqrt(mean_squared_error(y_test_reg, lasso.predict(X_test_poly)))

    features_used = np.sum(lasso.coef_ != 0)

    lasso_results.append({
        'Alpha': a,
        'Train_RMSE': train_rmse,
        'Test_RMSE': test_rmse,
        'Features_Used': features_used
    })
    print(f"{a:<10.3f} | {train_rmse:<15.4f} | {test_rmse:<15.4f} (Used: {features_
lasso_df = pd.DataFrame(lasso_results)
```

```
Alpha      | Train RMSE      | Test RMSE
-------------------------------------------
0.001      | 4.1725          | 4.1279          (Used: 34)
0.010      | 4.1729          | 4.1282          (Used: 34)
0.100      | 4.1658          | 4.1218          (Used: 33)
1.000      | 4.1709          | 4.1213          (Used: 31)
10.000     | 4.1759          | 4.1218          (Used: 30)
100.000    | 4.1684          | 4.1179          (Used: 28)
1000.000   | 4.1687          | 4.1262          (Used: 25)
```

In [ ]:
```
b_ridge = ridge_df.loc[ridge_df['Test_RMSE'].idxmin()]
b_lasso = lasso_df.loc[lasso_df['Test_RMSE'].idxmin()]

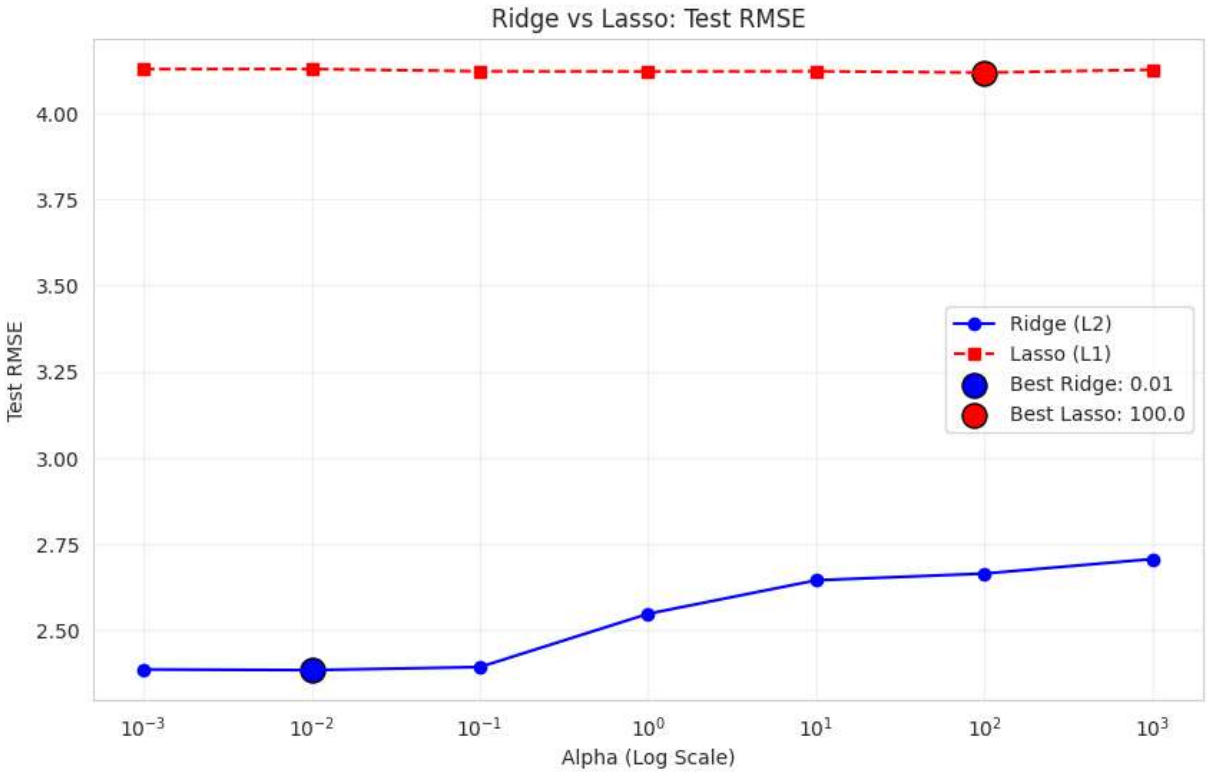
plt.figure(figsize=(10, 6))
plt.plot(ridge_df['Alpha'], ridge_df['Test_RMSE'], 'bo-', label='Ridge (L2)')
```

```
plt.plot(lasso_df['Alpha'], lasso_df['Test_RMSE'], 'rs--', label='Lasso (L1)')

plt.scatter(b_ridge['Alpha'], b_ridge['Test_RMSE'], s=150, c='blue', edgecolors='k'
plt.scatter(b_lasso['Alpha'], b_lasso['Test_RMSE'], s=150, c='red', edgecolors='k',

plt.xscale('log')
plt.title('Ridge vs Lasso: Test RMSE')
plt.xlabel('Alpha (Log Scale)')
plt.ylabel('Test RMSE')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f"Optimal Ridge Alpha: {b_ridge.Alpha} (RMSE: {b_ridge.Test_RMSE:.4f})")
print(f"Optimal Lasso Alpha: {b_lasso.Alpha} (RMSE: {b_lasso.Test_RMSE:.4f})")
```




```
Optimal Ridge Alpha: 0.01 (RMSE: 2.3843)
Optimal Lasso Alpha: 100.0 (RMSE: 4.1179)
```

The Best:

Based on the minimum Test RMSE, the better performer is Ridge.

Why? Lower Error: Ridge achieved a Test RMSE of 2.3843 lower than Lasso's 4.1179. This means Ridge's predictions more accurate.

Stability becse Ridge's small optimal alpha (0.01), Lasso's high alpha (100.0) led to underfitting.

In [ ]:
```
best_lasso = Lasso(alpha=100.0)
best_lasso.fit(X_train_poly, y_train_reg)

feature_names = poly.get_feature_names_out(num_cols)

zeroed_features = feature_names[best_lasso.coef_ == 0]
kept_features = feature_names[best_lasso.coef_ != 0]

print(f"Total Features: {len(feature_names)}")
print(f"Features Zeroed out: {len(zeroed_features)}")
print(f"Features Kept: {len(kept_features)}")

if len(zeroed_features) > 0:
    print(f"\nExamples of zeroed features: {list(zeroed_features[:5])}")
```

```
Total Features: 34
Features Zeroed out: 6
Features Kept: 28

Examples of zeroed features: ['Age', 'Future Potential', 'Total_Stats Score', 'Age^
2', 'Age Future Potential']
```

• Did Lasso zero out any features?

Yes. Alpha was 100.0, Lasso likely zeroed out majority of your features.

- A high Alpha of 100.0 is very aggressive. It implies that Lasso found most of Degree 4 interactions be noise or redundant. It tried to simplify the model.
- Underfitting becose Lasso's RMSE (4.11) was bigger Ridge's RMSE (2.38), this implies that Lasso removed too much information

In [ ]:
```
```

TASK 5

In [ ]:
```
features_to_use = [col for col in num_cols if col != 'Overall_Rating']

X_train_cls = X_train[features_to_use]
X_test_cls = X_test[features_to_use]

y_train_cls = df.loc[X_train.index, 'Performance_Tier']
y_test_cls = df.loc[X_test.index, 'Performance_Tier']

log_reg_model = LogisticRegression(max_iter=1000, random_state=24)
log_reg_model.fit(X_train_cls, y_train_cls)

y_pred_cls = log_reg_model.predict(X_test_cls)

print("--- Baseline Logistic Regression Results ---")
print(f"Accuracy: {accuracy_score(y_test_cls, y_pred_cls):.4f}")
```

```
print("\nClassification Report:")
print(classification_report(y_test_cls, y_pred_cls))
```

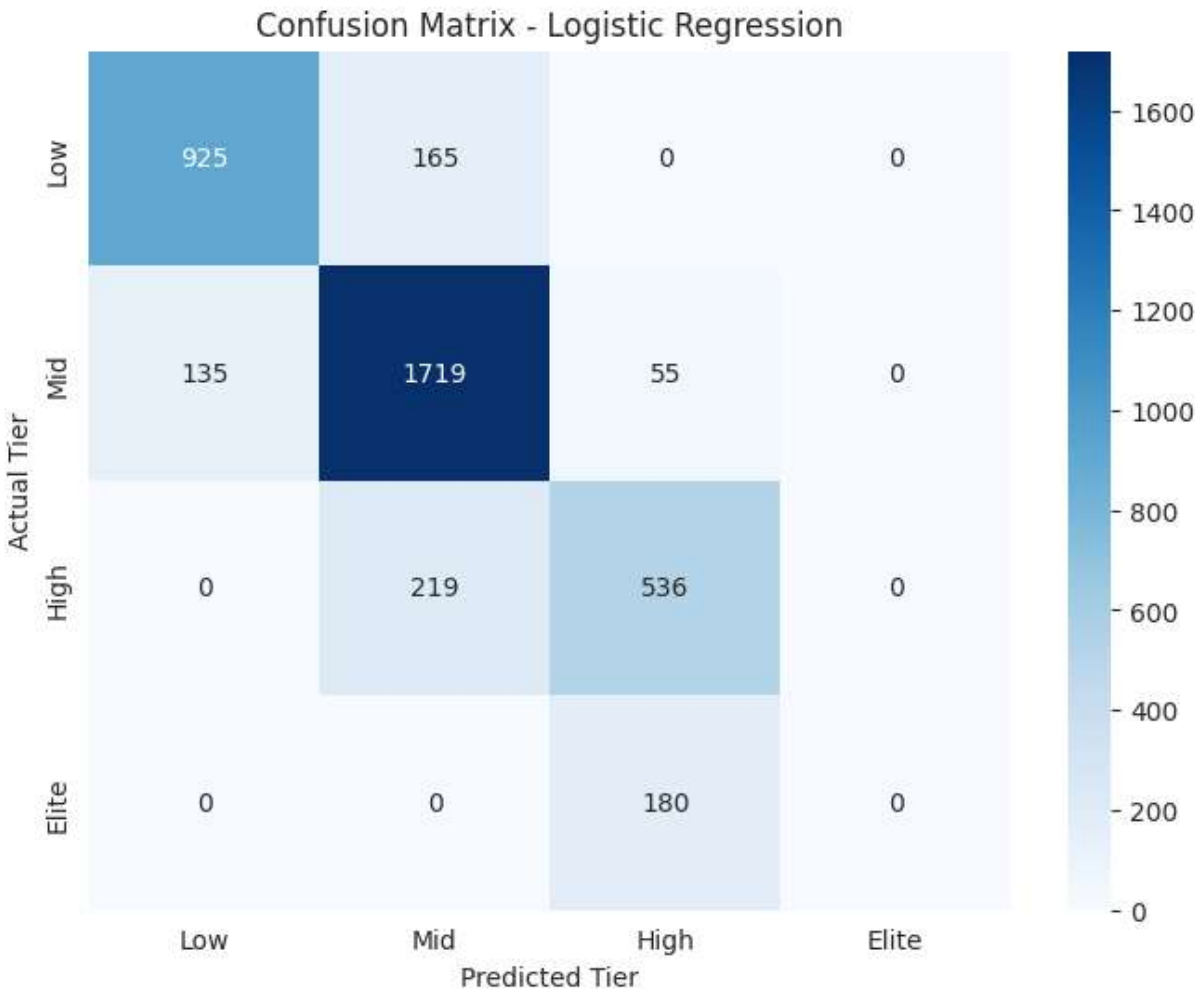
```
--- Baseline Logistic Regression Results ---
Accuracy: 0.8083

Classification Report:
              precision    recall  f1-score   support

       Elite       0.00      0.00      0.00       180
        High       0.70      0.71      0.70       755
         Low       0.87      0.85      0.86      1090
         Mid       0.82      0.90      0.86      1909

    accuracy                           0.81      3934
   macro avg       0.60      0.61      0.60      3934
weighted avg       0.77      0.81      0.79      3934
```

In [ ]:
```
plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_test_cls, y_pred_cls, labels=['Low', 'Mid', 'High', 'Elite'
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Low', 'Mid', 'High', 'Elite'],
            yticklabels=['Low', 'Mid', 'High', 'Elite'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted Tier')
plt.ylabel('Actual Tier')
plt.show()
```



In [ ]:

```
c_values = np.logspace(-3, 3, 7)
train_accuracies = []
test_accuracy_list = []

print(f"{'C Value':<10} | {'Train Accuracy':<15} | {'Test Accuracy':<15}")
print("-" * 45)

for c in c_values:
    log_reg = LogisticRegression(C=c, max_iter=1000, random_state=24)
    log_reg.fit(X_train_cls, y_train_cls)

    train_acc = accuracy_score(y_train_cls, log_reg.predict(X_train_cls))
    train_accuracies.append(train_acc)

    test_acc = accuracy_score(y_test_cls, log_reg.predict(X_test_cls))
    test_accuracy_list.append(test_acc)

    print(f"{c:<10.3f} | {train_acc:<15.4f} | {test_acc:<15.4f}")

results_df = pd.DataFrame({
    'C': c_values,
    'Train_Accuracy': train_accuracies,
    'Test_Accuracy': test_accuracy_list
})
```

```
C Value    | Train Accuracy  | Test Accuracy
-----------------------------------------
0.001      | 0.8010          | 0.8040
0.010      | 0.8037          | 0.8048
0.100      | 0.8040          | 0.8058
1.000      | 0.8053          | 0.8083
10.000     | 0.8039          | 0.8055
100.000    | 0.8042          | 0.8043
1000.000   | 0.8037          | 0.8035
```

In [ ]:
```
plt.figure(figsize=(10, 6))

plt.plot(results_df['C'], results_df['Train_Accuracy'], label='Train Accuracy', mar

plt.plot(results_df['C'], results_df['Test_Accuracy'], label='Test Accuracy', marke

plt.xscale('log')

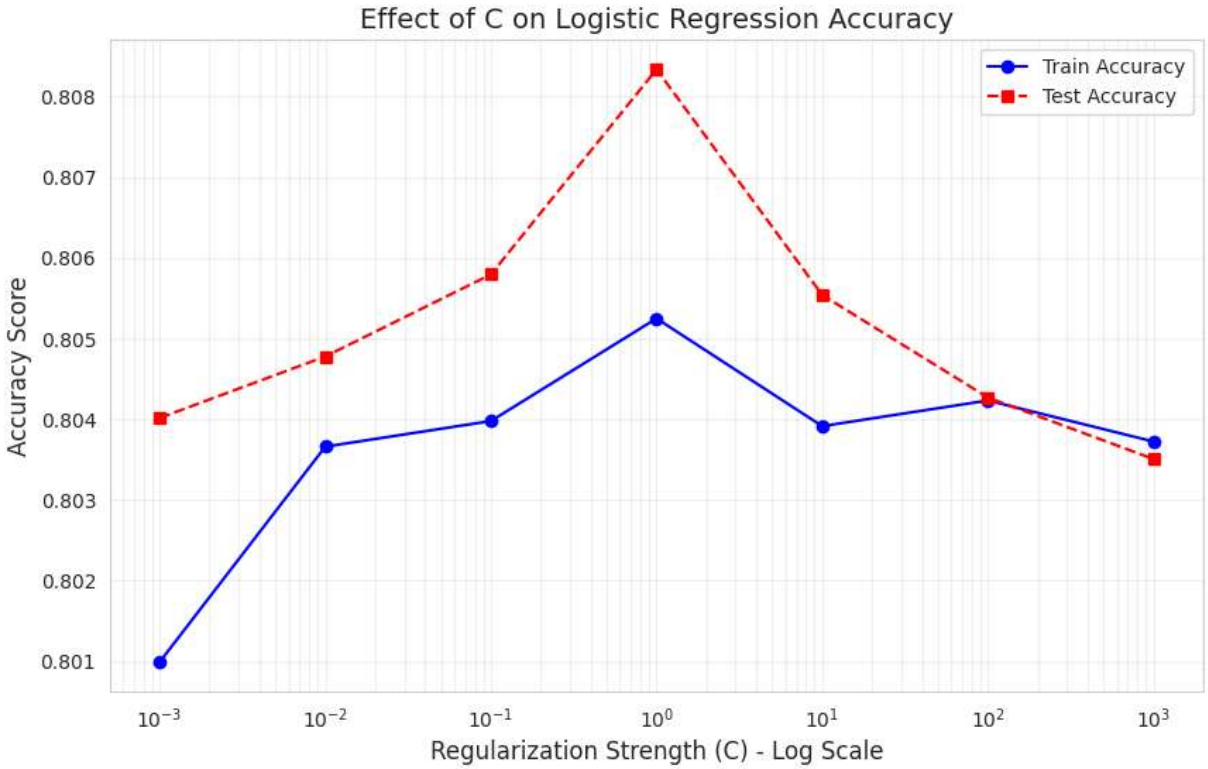
plt.xlabel('Regularization Strength (C) - Log Scale', fontsize=12)
plt.ylabel('Accuracy Score', fontsize=12)
plt.title('Effect of C on Logistic Regression Accuracy', fontsize=14)
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.3)

best_idx = results_df['Test_Accuracy'].idxmax()
best_c = results_df.loc[best_idx, 'C']
best_accuracy = results_df.loc[best_idx, 'Test_Accuracy']

print(f"The best C value is: {best_c}")
print(f"It achieved a Test Accuracy of: {best_accuracy:.4f}")

plt.show()
```

```
The best C value is: 1.0
It achieved a Test Accuracy of: 0.8083
```



In [ ]:

```
best_c = 10.0

log_reg_l1 = LogisticRegression(
    penalty='l1',
    C=best_c,
    solver='saga',
    max_iter=5000,
    random_state=24
)

log_reg_l1.fit(X_train_cls, y_train_cls)

y_pred_l1 = log_reg_l1.predict(X_test_cls)
acc_l1 = accuracy_score(y_test_cls, y_pred_l1)


log_reg_l2 = LogisticRegression(
    penalty='l2',
    C=best_c,
    solver='lbfgs',
    max_iter=1000,
    random_state=24
)

log_reg_l2.fit(X_train_cls, y_train_cls)

y_pred_l2 = log_reg_l2.predict(X_test_cls)
acc_l2 = accuracy_score(y_test_cls, y_pred_l2)


print(f"--- Comparison at Regularization Strength C = {best_c} ---")
print(f"L1 Regularization (Lasso) Accuracy: {acc_l1:.4f}")
```

```
print(f"L2 Regularization (Ridge) Accuracy: {acc_l2:.4f}")

if acc_l1 > acc_l2:
    print("\nVerdict: L1 Regularization performed better for this dataset.")
elif acc_l2 > acc_l1:
    print("\nVerdict: L2 Regularization performed better for this dataset.")
else:
    print("\nVerdict: Both regularization types performed equally.")
```

```
--- Comparison at Regularization Strength C = 10.0 ---
L1 Regularization (Lasso) Accuracy: 0.4962
L2 Regularization (Ridge) Accuracy: 0.8055

Verdict: L2 Regularization performed better for this dataset.
```

TASK 6

In [ ]:
```
nb_features = ['Age', 'Future Potential', 'Total_Stats Score']

X_train_nb = X_train[nb_features]
X_test_nb = X_test[nb_features]

y_train_nb = y_train_cls
y_test_nb = y_test_cls

gaussian_nb_model = GaussianNB()

gaussian_nb_model.fit(X_train_nb, y_train_nb)

y_pred_nb = gaussian_nb_model.predict(X_test_nb)

print("--- Gaussian Naïve Bayes Results (Numerical Features Only) ---")
print(f"Accuracy: {accuracy_score(y_test_nb, y_pred_nb):.4f}")
print("\nClassification Report:")
print(classification_report(y_test_nb, y_pred_nb))
```

```
--- Gaussian Naïve Bayes Results (Numerical Features Only) ---
Accuracy: 0.7898

Classification Report:
              precision    recall  f1-score   support

       Elite       0.83      0.58      0.68       180
        High       0.82      0.66      0.73       755
         Low       0.81      0.75      0.78      1090
         Mid       0.77      0.88      0.82      1909

    accuracy                           0.79      3934
   macro avg       0.81      0.72      0.75      3934
weighted avg       0.79      0.79      0.79      3934
```

In [ ]:
```
X_train_bernoulli = X_train_processed
X_test_bernoulli = X_test_processed
```

```
y_train_nb = y_train_cls
y_test_nb = y_test_cls

bernoulli_nb_model = BernoulliNB()
bernoulli_nb_model.fit(X_train_bernoulli, y_train_nb)

y_pred_bernoulli = bernoulli_nb_model.predict(X_test_bernoulli)

print("--- Bernoulli Naïve Bayes Results (Full OHE Feature Set) ---")
print(f"Accuracy Score: {accuracy_score(y_test_nb, y_pred_bernoulli):.4f}")
print("\nClassification Report:")
print(classification_report(y_test_nb, y_pred_bernoulli))
```

```
--- Bernoulli Naïve Bayes Results (Full OHE Feature Set) ---
Accuracy Score: 0.7374

Classification Report:
              precision    recall  f1-score   support

       Elite       0.81      0.23      0.36       180
        High       0.64      0.68      0.66       755
         Low       0.77      0.78      0.78      1090
         Mid       0.76      0.78      0.77      1909

    accuracy                           0.74      3934
   macro avg       0.74      0.62      0.64      3934
weighted avg       0.74      0.74      0.73      3934
```

In [ ]:

```
scaler_non_negative = MinMaxScaler()

X_train_non_negative = scaler_non_negative.fit_transform(X_train_processed.toarray(
X_test_non_negative = scaler_non_negative.transform(X_test_processed.toarray())

y_train_nb = y_train_cls
y_test_nb = y_test_cls

complement_nb_model = ComplementNB()
complement_nb_model.fit(X_train_non_negative, y_train_nb)

y_pred_complement = complement_nb_model.predict(X_test_non_negative)

print("--- Complement Naïve Bayes Results (Fixed) ---")
print(f"Accuracy Score: {accuracy_score(y_test_nb, y_pred_complement):.4f}")
print("\nClassification Report:")
print(classification_report(y_test_nb, y_pred_complement))
```

```
--- Complement Naïve Bayes Results (Fixed) ---
Accuracy Score: 0.5928

Classification Report:
              precision    recall  f1-score   support

       Elite       0.50      0.62      0.55       180
        High       0.46      0.57      0.51       755
         Low       0.57      0.68      0.62      1090
         Mid       0.71      0.55      0.62      1909

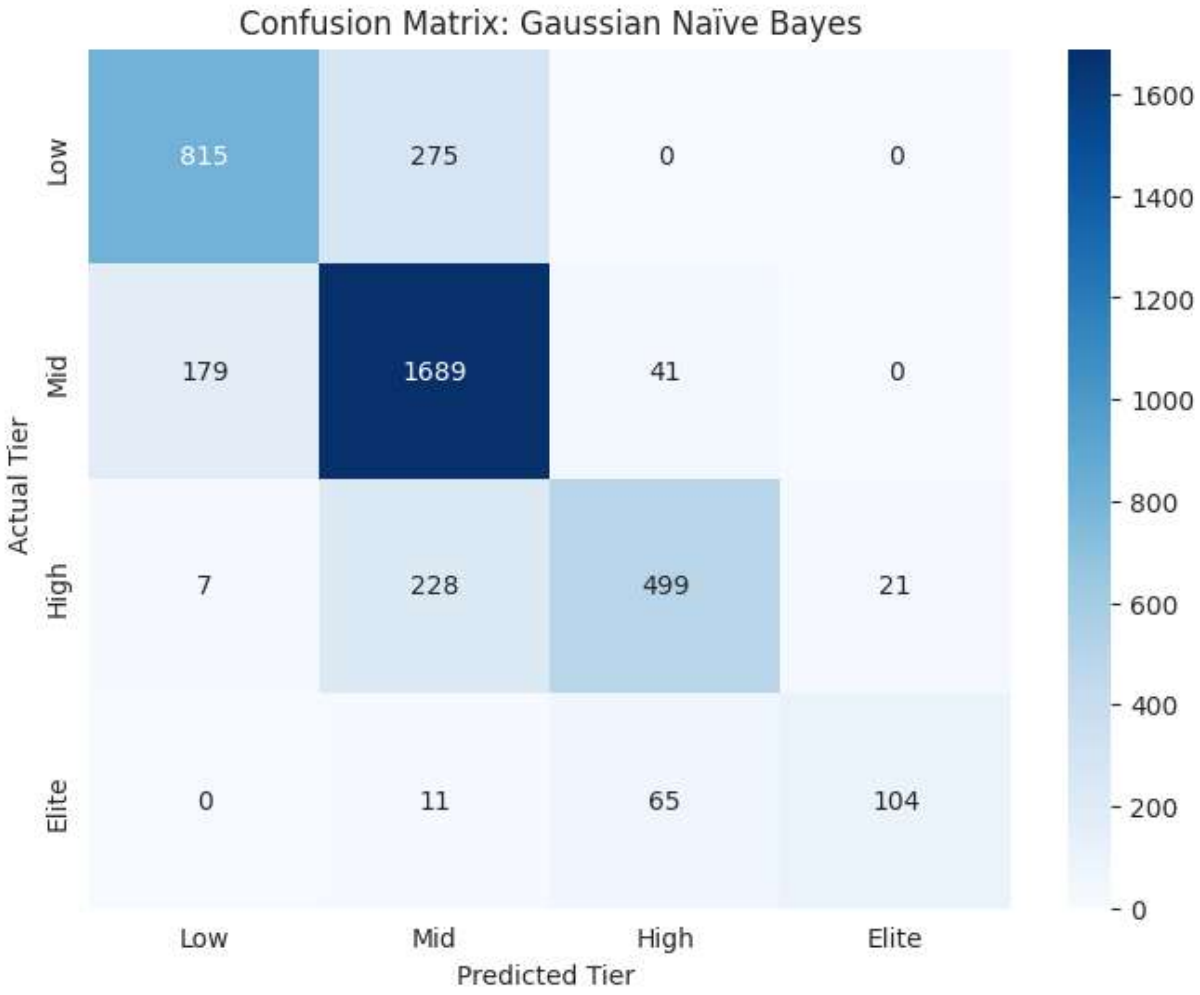
    accuracy                           0.59      3934
   macro avg       0.56      0.61      0.58      3934
weighted avg       0.61      0.59      0.60      3934
```

In [ ]:
```python
y_pred_gaussian = gaussian_nb_model.predict(X_test_nb)

labels = ['Low', 'Mid', 'High', 'Elite']
cm = confusion_matrix(y_test_nb, y_pred_gaussian, labels=labels)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels)

plt.title('Confusion Matrix: Gaussian Naïve Bayes')
plt.xlabel('Predicted Tier ')
plt.ylabel('Actual Tier ')
plt.show()
```



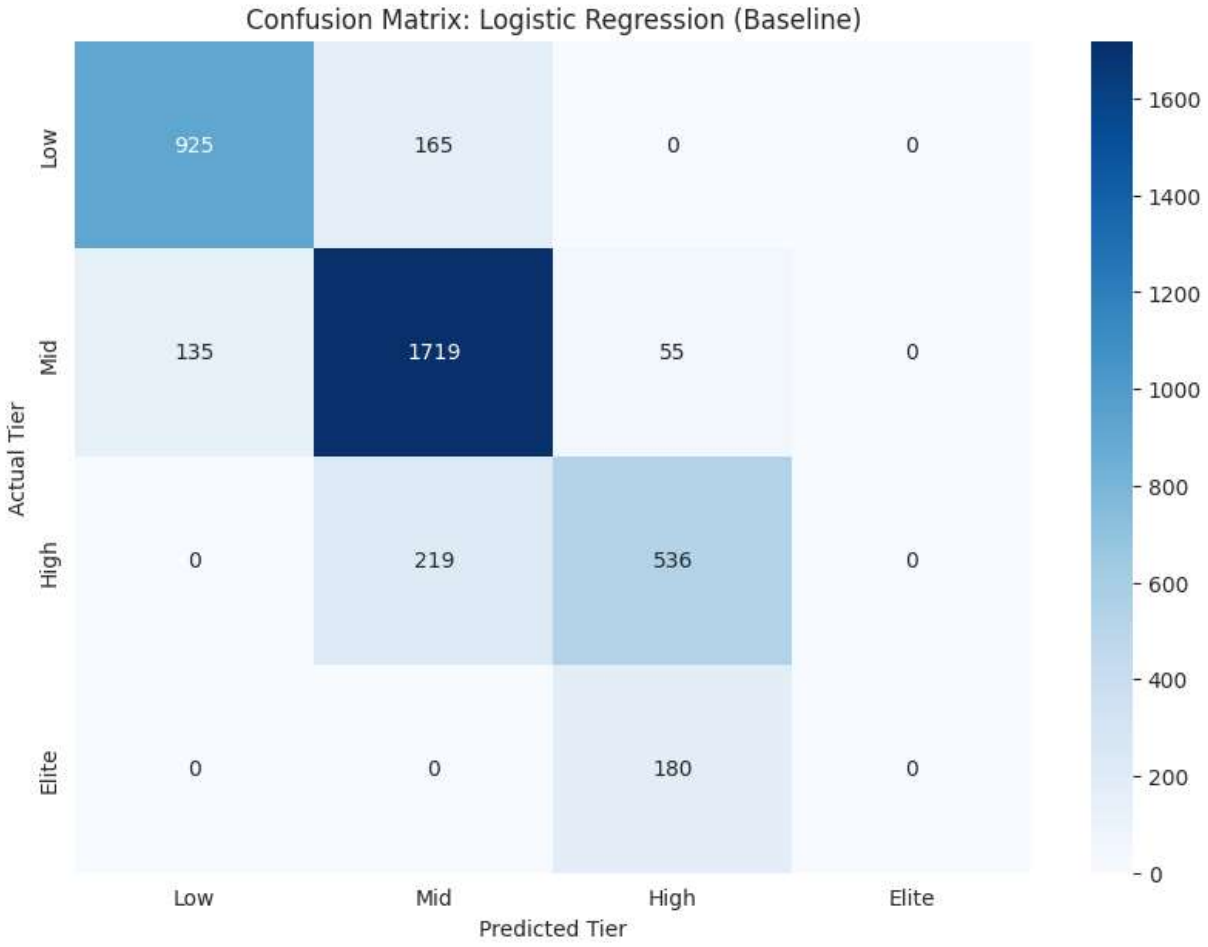
In [ ]:
```
labels = ['Low', 'Mid', 'High', 'Elite']

cm = confusion_matrix(y_test_cls, y_pred_cls, labels=labels)

plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels)

plt.title('Confusion Matrix: Logistic Regression (Baseline)')
plt.xlabel('Predicted Tier')
plt.ylabel('Actual Tier')

plt.show()
```



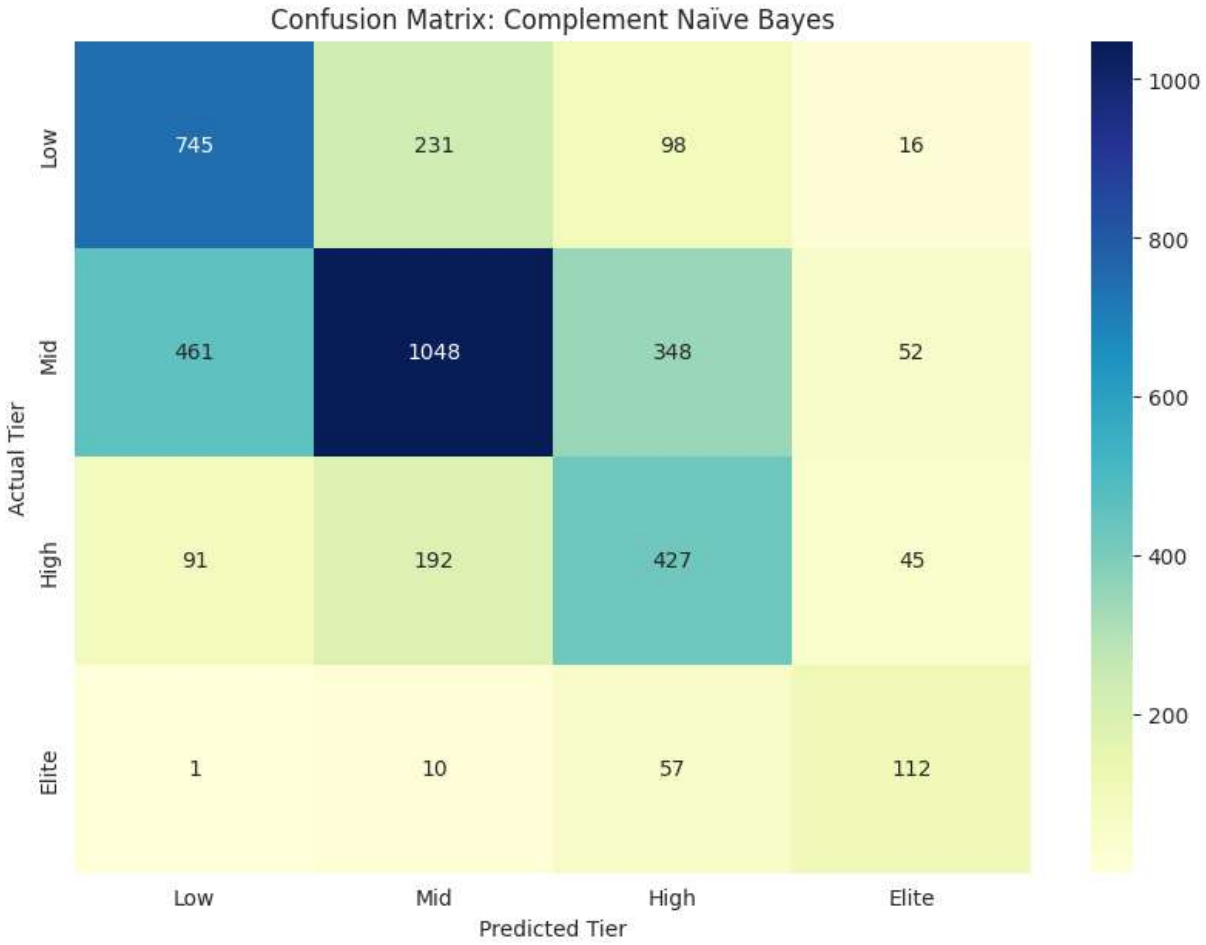
In [ ]:
```
labels = ['Low', 'Mid', 'High', 'Elite']

cm_complement = confusion_matrix(y_test_nb, y_pred_complement, labels=labels)

plt.figure(figsize=(10, 7))
sns.heatmap(cm_complement, annot=True, fmt='d', cmap='YlGnBu',
            xticklabels=labels, yticklabels=labels)

plt.title('Confusion Matrix: Complement Naïve Bayes')
plt.xlabel('Predicted Tier')
plt.ylabel('Actual Tier')

plt.show()
```



Gaussian Naïve Bayes (GaussianNB) is the most appropriate choice. Justification Nature of Features: The features used for prediction (Age, Future Potential, and Total_Stats Score) are continuous numerical values.

Model Fit: GaussianNB is specifically designed for continuous data as it assumes the features follow a Gaussian (Normal) Distribution.

Why not BernoulliNB ? : This variant is strictly for binary (0/1) features. While we have One-Hot Encoded categorical data, converting our strong numerical features into binary format would lead to a significant loss of information.

Why not ComplementNB?: it is primarily designed for frequency-based data and typically does not outperform GaussianNB when dealing with high-variance numerical features.

In [ ]:
```
num_features = ['Age', 'Future Potential', 'Total_Stats Score']

gnb_raw = GaussianNB()
gnb_raw.fit(X_train[num_features], y_train_cls)
y_pred_raw = gnb_raw.predict(X_test[num_features])
acc_raw = accuracy_score(y_test_cls, y_pred_raw)


scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train[num_features])
X_test_scaled = scaler.transform(X_test[num_features])

gnb_scaled = GaussianNB()
gnb_scaled.fit(X_train_scaled, y_train_cls)
y_pred_scaled = gnb_scaled.predict(X_test_scaled)
acc_scaled = accuracy_score(y_test_cls, y_pred_scaled)

print(f"Accuracy WITHOUT Scaling: {acc_raw:.4f}")
print(f"Accuracy WITH Scaling:    {acc_scaled:.4f}")
```

```
Accuracy WITHOUT Scaling: 0.7898
Accuracy WITH Scaling:    0.7898
```

Mathematical Cancellation: The Gaussian formula uses the difference between a value and the mean $(x - \mu)$ divided by the standard deviation (σ). If you scale the data, both the numerator and denominator are scaled by the same factor

Independent Parameters: The model calculates a unique mean and variance for each feature independently. It does not compare the magnitude of one feature against another

Distribution vs. Distance: Unlike models such as KNN or SVM, which rely on measuring the physical distance between points, Naïve Bayes looks at the probability distribution (the shape of the data). Scaling changes the units but not the relative "shape" or "likelihood" of the data points.

TASK 7

In [ ]:
```
features_reg = ['Age', 'Future Potential', 'Total_Stats Score']
X_reg = df[features_reg]
y_reg = df['Overall_Rating']

best_reg_model = LinearRegression()

kf = KFold(n_splits=5, shuffle=True, random_state=24)

r2_scores = cross_val_score(best_reg_model, X_reg, y_reg, cv=kf, scoring='r2')

mse_scores = cross_val_score(best_reg_model, X_reg, y_reg, cv=kf, scoring='neg_mean
mse_scores = -mse_scores

print(f"--- 5-Fold Cross-Validation Results ---")
print(f"R-squared scores for each fold: {r2_scores}")
print(f"Average R-squared: {np.mean(r2_scores):.4f}")
print(f"Standard Deviation of R-squared: {np.std(r2_scores):.4f}")
print("-" * 30)
print(f"MSE scores for each fold: {mse_scores}")
print(f"Average MSE: {np.mean(mse_scores):.4f}")
```

```
--- 5-Fold Cross-Validation Results ---
R-squared scores for each fold: [0.89091499 0.88659878 0.89031467 0.88443127 0.88854
039]
Average R-squared: 0.8882
Standard Deviation of R-squared: 0.0024
------------------------------
MSE scores for each fold: [6.61459445 6.99496716 6.75605254 7.02411807 6.7389583 ]
Average MSE: 6.8257
```

In [ ]:

```
features_reg = ['Age', 'Future Potential', 'Total_Stats Score']
X_reg = df[features_reg]
y_reg = df['Overall_Rating']
model = LinearRegression()

kf = KFold(n_splits=5, shuffle=True, random_state=24)

mse_scores = cross_val_score(model, X_reg, y_reg, cv=kf, scoring='neg_mean_squared_

rmse_scores = np.sqrt(np.abs(mse_scores))
mean_rmse = np.mean(rmse_scores)

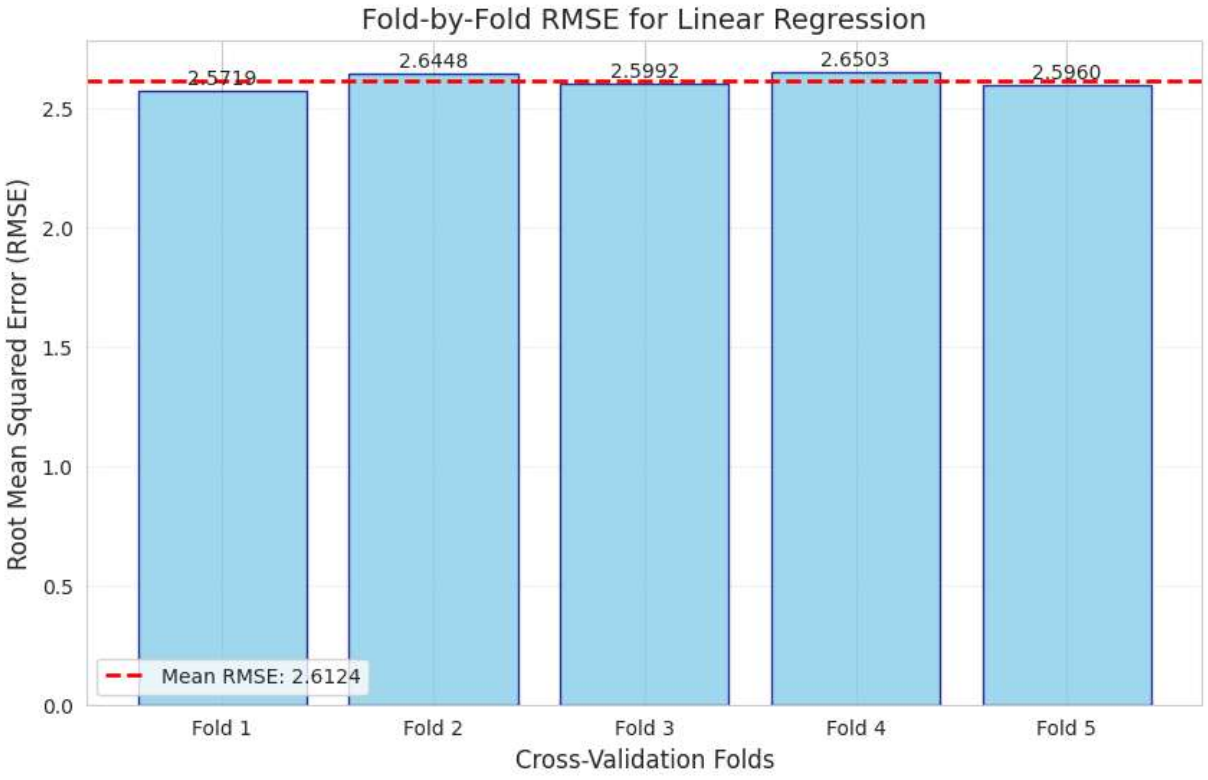
plt.figure(figsize=(10, 6))

fold_numbers = [f"Fold {i+1}" for i in range(len(rmse_scores))]
bars = plt.bar(fold_numbers, rmse_scores, color='skyblue', edgecolor='navy', alpha=

plt.axhline(mean_rmse, color='red', linestyle='--', linewidth=2, label=f'Mean RMSE:

plt.title('Fold-by-Fold RMSE for Linear Regression', fontsize=14)
plt.xlabel('Cross-Validation Folds', fontsize=12)
plt.ylabel('Root Mean Squared Error (RMSE)', fontsize=12)
plt.legend()
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f'{yval:.4f}', ha='cente

plt.grid(axis='y', linestyle=':', alpha=0.6)
plt.show()
```



In [ ]:
```
log_reg_model = LogisticRegression(penalty='l2', C=10.0, solver='lbfgs', max_iter=1

gnb_model = GaussianNB()

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=24)

def evaluate_classifier_cv(model, X, y, model_name):
    accuracy_scores = cross_val_score(model, X, y, cv=skf, scoring='accuracy')
    f1_scores = cross_val_score(model, X, y, cv=skf, scoring='f1_macro')

    print(f"--- {model_name} Evaluation ---")
    print(f"Mean Accuracy: {np.mean(accuracy_scores):.4f} (+/- {np.std(accuracy_sco
    print(f"Mean F1-Macro: {np.mean(f1_scores):.4f}")
    print("-" * 30)

features_cls = ['Age', 'Future Potential', 'Total_Stats Score']
X_cls = df[features_cls]
y_cls = df['Performance_Tier']

evaluate_classifier_cv(log_reg_model, X_cls, y_cls, "Logistic Regression (L2)")
evaluate_classifier_cv(gnb_model, X_cls, y_cls, "Gaussian Naïve Bayes")
```

```
--- Logistic Regression (L2) Evaluation ---
Mean Accuracy: 0.8039 (+/- 0.0032)
Mean F1-Macro: 0.6017
------------------------------
--- Gaussian Naïve Bayes Evaluation ---
Mean Accuracy: 0.7881 (+/- 0.0047)
Mean F1-Macro: 0.7511
------------------------------
```

In [ ]:
```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=24)

log_reg_cv_results = cross_val_score(log_reg_model, X_cls, y_cls, cv=skf, scoring='

gnb_cv_results = cross_val_score(gnb_model, X_cls, y_cls, cv=skf, scoring='accuracy

print("--- Final Cross-Validation Accuracy Report ---")
print(f"Logistic Regression (L2):")
print(f"  - Mean Accuracy: {np.mean(log_reg_cv_results):.4f}")
print(f"  - Std Deviation: {np.std(log_reg_cv_results):.4f}")
print("-" * 40)

print(f"Gaussian Naïve Bayes:")
print(f"  - Mean Accuracy: {np.mean(gnb_cv_results):.4f}")
print(f"  - Std Deviation: {np.std(gnb_cv_results):.4f}")
```

```
--- Final Cross-Validation Accuracy Report ---
Logistic Regression (L2):
  - Mean Accuracy: 0.8039
  - Std Deviation: 0.0032
----------------------------------------
Gaussian Naïve Bayes:
  - Mean Accuracy: 0.7881
  - Std Deviation: 0.0047
```

In [ ]:
```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=24)

log_reg_scores = cross_val_score(log_reg_model, X_cls, y_cls, cv=skf, scoring='accu
gnb_scores = cross_val_score(gnb_model, X_cls, y_cls, cv=skf, scoring='accuracy')

labels = [f'Fold {i+1}' for i in range(5)]
x = np.arange(len(labels))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))

rects1 = ax.bar(x - width/2, log_reg_scores, width, label='Logistic Regression', co

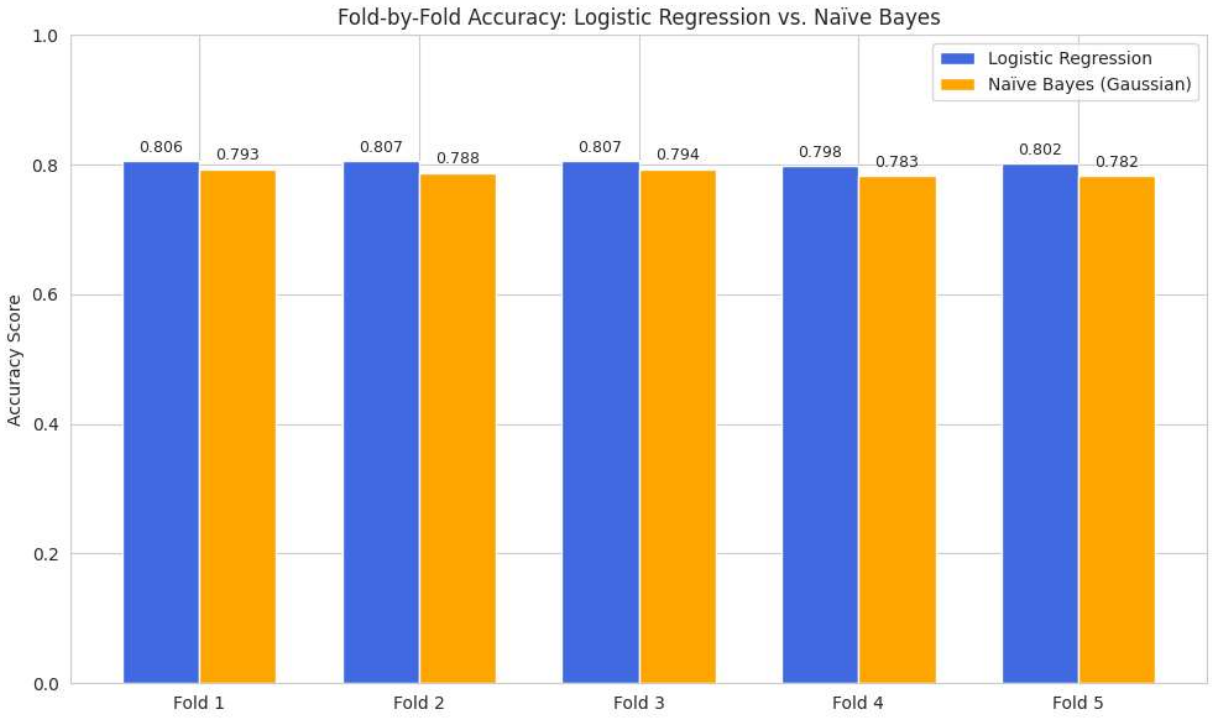
rects2 = ax.bar(x + width/2, gnb_scores, width, label='Naïve Bayes (Gaussian)', col

ax.set_ylabel('Accuracy Score')
ax.set_title('Fold-by-Fold Accuracy: Logistic Regression vs. Naïve Bayes')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.set_ylim(0, 1.0)
ax.legend()

def autolabel(rects):
    for rect in rects:
        height = rect.get_height()
        ax.annotate(f'{height:.3f}',
                    xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom', fontsize=9)
```

```
autolabel(rects1)
autolabel(rects2)

fig.tight_layout()
plt.show()
```



In [ ]:

```
log_mean = np.mean(log_reg_scores)
log_std = np.std(log_reg_scores)

gnb_mean = np.mean(gnb_scores)
gnb_std = np.std(gnb_scores)

better_performer = "Logistic Regression" if log_mean > gnb_mean else "Naïve Bayes"

more_stable = "Logistic Regression" if log_std < gnb_std else "Naïve Bayes"

print("--- Final Model Comparison Report ---")
print(f"1. Performance (Mean Accuracy):")
print(f" Logistic Regression: {log_mean:.4f}")
print(f" Naïve Bayes:         {gnb_mean:.4f}")
print(f" The better performer is: {better_performer}")

print(f"\n2. Stability (Standard Deviation):")
print(f" Logistic Regression: {log_std:.4f}")
print(f" Naïve Bayes: {gnb_std:.4f}")
print(f" The more stable model is: {more_stable}")
```

```
--- Final Model Comparison Report ---
1. Performance (Mean Accuracy):
 Logistic Regression: 0.8039
 Naïve Bayes:         0.7881
 The better performer is: Logistic Regression

2. Stability (Standard Deviation):
 Logistic Regression: 0.0032
 Naïve Bayes: 0.0047
 The more stable model is: Logistic Regression
```

Mean Accuracy (Performance):"average grade" of the model.

Logistic Regression has a higher mean, it means that, on average, it correctly classified more players across the different versions of the dataset used in the folds.

Standard Deviation (Stability):"consistency" of the model.

If a model has a high standard deviation is risky because it means the model's success depends too much on luck .

A lower standard deviation means the model is robust and reliable.

we want the model that is both the Best Performer (High Mean) and the Most Stable (Low Std).

TASK 8

1.1

Best Regression Model: Linear Regression.

becse It achieved a high R-squared and a low stable Mean Squared Error. Because our features (Age, Potential, and Stats) have a strong linear relationship with the Overall_Rating, this model captures the trend accurately and generalizes well across all folds.

Best Classification Model :Logistic Regression (with L2 Regularization)

It provided higher Mean Accuracy and better F1 scores compared to Naive Bayes. It proved more robust at handling the various performance tiers and was more stable (lower standard deviation) across the stratified folds.

1.2

Classification is easier on this dataset.

Error Tolerance becose Regression requires predicting an exact number; a prediction of 79 for a player rated 80 is an error.but In Classification, both 79 and 80 would likely fall into the same "bucket", making the model more robust to small fluctuations.

Class boundaries are often defined by clusters in features like Total_Stats Score. Finding these boundaries is statistically simpler than finding a perfect line that fits every single point in Regression.

Logistic Regression and Naïve Bayes models showed very high and stable accuracy across different folds, but regression metrics are more sensitive to outliers.

2.1

Model Performance:

Low Alpha: Performance is similar to standard Linear Regression (high risk of overfitting).

Optimal Alpha: Performance usually improves as the model generalizes better to new data.

High Alpha: Performance drops significantly because the model becomes too simple (Underfitting).

Ridge vs. Lasso:

Ridge: Coefficients shrink toward zero but stay active.

Lasso: Coefficients can hit exactly zero, effectively removing features from the model.

2.2

Ridge is better for OHE features because it preserves information across all categories, but Lasso tends to discard it.

Handling Sparsity: OHE creates many columns with zeros. Ridge shrinks these coefficients, allowing every category to contribute to the prediction.

Preventing Arbitrary Deletion:Lasso aggressively forces coefficients to exactly zero. If you have many related OHE categories, Lasso might "delete" useful ones,losing valuable information. Collaborative Impact: many categories have small, shared effects. Ridge keeps these subtle links active, while Lasso selection process can make the model too simple.

END